Ψηφιακή Επεξεργασία Εικόνας

Εργασία 3

Image segmentation

Τμήμα Ηλεκτρολόγων Μηχανικών και Μηχανικών Υπολογιστών ΑΠΘ

8ο εξάμηνο

Θεσσαλονίκη, Ιούνιος 2025

Υπεύθυνη εργασίας: Μανωλίδου Ανατολή

Email: amanolid@ece.auth.gr

ΑΕΜ: 10874

Περιεχόμενα

1 Εισαγωγή ……3

2 Image to Graph ……3

2.1 image_to_graph.py……3

3 Image Segmentation……4

3.1 spectral_clustering.py……4

3.2 n_cuts.py……6

4 Demos……9

4.1 demo1.py……9

4.2 demo2.py……10

4.3 demo3a.py……14

4.4 dem3b.py……18

4.5 demo3c.py……19

1 Εισαγωγή

Στην παρούσα εργασία, μελετάμε την τεχνική της ομαδοποίησης για την κατάτμηση εικόνων, μέσω των μεθόδων spectral clustering και normalized cuts, με την αναδρομική και τη μη-αναδρομική μέθοδο. Οι μέθοδοι αυτές βασίζονται στην αναπαράσταση της εικόνας σαν γράφο.

2 Image to Graph

Το πρώτο τμήμα της εργασίας ασχολείται με την αναπαράσταση εικόνων ως γράφους. Συγκεκριμένα, η συνάρτηση είναι υπεύθυνη για τη μετατροπή μιας έγχρωμης εικόνας σε γράφο, μέσω της δημιουργίας ενός πίνακα συσχετίσεων. Κάθε pixel της εικόνας το θεωρούμε ως κόμβο του γράφου, ενώ το βάρος μεταξύ δύο κόμβων ορίζεται ως:

$$\boldsymbol{A}(i,j) \;=\; 1/e^{d(i,j)}$$

όπου $d(i, j)$ η Ευκλείδεια απόσταση της φωτεινότητας των καναλιών μεταξύ του i-οστού και του j-οστού pixel.

2.1 image_to_graph.py

Παρακάτω παρατίθεται ο υλοποιημένος κώδικας της συνάρτησης image_to_graph:

```
def image_to_graph(
        img_array: np.ndarray,
) -> np.ndarray:

    #Checking if the input image array is 3-dimensional
    if img_array.ndim != 3:
        raise ValueError("Input image array must be 3-dimensional (M, N, C)")

    M, N, C = img_array.shape

    #Reshaping the image array to a 2D array of shape (MN, C)
    array = img_array.reshape(M * N, C)

    #Calculating the euclidean distance between the channels
    diff = array[:, np.newaxis, :] - array[np.newaxis, :, :]
```

```
    dist = np.sqrt(np.sum(diff ** 2, axis=-1))

    affinity_mat = 1 / np.exp(dist)

    return affinity_mat
```

Η είσοδος της συνάρτησης είναι ένας τρισδιάστατος πίνακας img_array, διαστάσεων (M, N, C), που αναπαριστά μια εικόνα με C κανάλια. Στην αρχή κάνουμε έναν έλεγχο για να σιγουρευτούμε πως έχουμε εικόνα τριών διαστάσεων. Στη συνέχεια, μετασχηματίζουμε το πίνακα εισόδου σε δισδιάστατη μορφή (MN, C), όπου κάθε γραμμή αντιστοιχεί σε ένα pixel. Έπειτα, υπολογίζουμε, την ευκλείδεια απόσταση μεταξύ όλων των ζευγών pixels και ο affinity πίνακας σχηματίζεται με το τρόπο που αναφέραμε παραπάνω. Το αποτέλεσμα είναι ένας πλήρως συνδεδεμένος γράφος, το οποίο και επιστρέφει η συνάρτηση.

3 Image Segmentation

Η τεχνική της κατάτμησης εικόνας (image segmentation) στοχεύει στη διαίρεση της εικόνας σε επιμέρους περιοχές με κοινά χαρακτηριστικά. Κάθε τέτοια περιοχή αναμένεται να εμφανίζει ομοιογένεια ως προς την ένταση, την υφή ή άλλα οπτικά γνωρίσματα.

3.1 spectral_clustering.py

Η τεχνική $spectral\ clustering$ εφαρμόζεται στην παρούσα εργασία για τον διαχωρισμό της εικόνας σε συνεκτικές περιοχές (clusters), με βάση τη φασματική ανάλυση του γράφου που προκύπτει από την εικόνα. Η βασική ιδέα είναι να θεωρήσουμε την εικόνα ως γράφο, όπου κάθε pixel αποτελεί έναν κόμβο.

Παρακάτω παρατίθεται ο υλοποιημένος κώδικας της συνάρτησης $spectral_clustering$:

```
def spectral_clustering(
        affinity_mat: np.ndarray,
        k: int
) -> np.ndarray:

    #Calculate Laplacian matrix
    D = np.diag(np.sum(affinity_mat, axis=1))
    L = D - affinity_mat
```

```
    #Finding the k smallest eigenvalues and their corresponding eigenvectors
using spicy.sparse.linalg.eigs
    eigenvalues, eigenvectors = eigsh(L, k = k, which='SM',
return_eigenvectors=True)

    #U matrix has as columns the eigenvectors
    U = eigenvectors

    #Applying k-means algorithm

    #Setting the random_state parameter to 1, to ensure reproducibility across
experiments.
    kmeans = KMeans(n_clusters=k, random_state=1)

    #U refers to the input data matrix
    kmeans.fit(U)

    #Obtain the cluster labels for each input data point
    cluster_idx = kmeans.labels_

    return cluster_idx
```

Αρχικά, υπολογίζεται ο Λαπλασιανός πίνακας (L) ως η διαφορά μεταξύ του διαγώνιου πίνακα βαθμών (D) και του πίνακα γειτνίασης (W). Αυτό γίνεται με χρήση των συναρτήσεων $np.sum$ και $np.diag$.

Στη συνέχεια, με τη βοήθεια της $eigsh$, εξάγονται τα k ιδιοδιανύσματα που αντιστοιχούν στις μικρότερες ιδιοτιμές του L. Πιο συγκεκριμένα, όσο αφορά τις επιλογές για τα ορίσματα της $eigsh$, έχουμε ως έχει:

- **k = k**: Ο αριθμός των ιδιοτιμών/ιδιοδιανυσμάτων που θα υπολογιστούν.
- **which='SM'**: Η επιλογή 'SM' σημαίνει *Smallest Magnitude*, έγινε καθώς επιθυμούμε να υπολογίσουμε τις μικρότερες σε μέτρο ιδιοτιμές.
- **return_eigenvectors=True**: Ζητάει να επιστραφούν και τα αντίστοιχα ιδιοδιανύσματα

Τα ιδιοδιανύσματα αυτά σχηματίζουν τον πίνακα U. Έπειτα, όπως μας υποδείχθηκε εφαρμόζουμε τον αλγόριθμο $K - Means$ με τη βοήθεια της $Kmeans$ συνάρτησης

Τέλος, με τη χρήση του έτοιμου αλγορίθμου $KMeans$, πραγματοποιείται ομαδοποίηση στον χώρο των ιδιοδιανυσμάτων και συγκεκριμένα σε k περιοχές. Ορίζοντας $random_state = 1$ εξασφαλίζουμε επαναληψιμότητα στα αποτελέσματα. Η συνάρτηση επιστρέφει τις τελικές ετικέτες ταξινόμησης των pixels σε clusters.

3.2 n_cuts.py

Πρώτη συνάρτηση είναι n_cuts, αυτή υλοποιεί τη μη-αναδρομική εκδοχή της μεθόδου normalized cuts για την ομαδοποίηση των κόμβων ενός γράφου σε k ομάδες.

Παρακάτω παρατίθεται ο υλοποιημένος κώδικας της συνάρτησης n_cuts:

```
def n_cuts(
        affinity_mat: np.ndarray,
        k: int
) -> np.ndarray:

        #Calculate Laplacian matrix
        D = np.diag(np.sum(affinity_mat, axis=1))
        L = D - affinity_mat

        #Solving Lx=λDx
        eigenvalues, eigenvectors = eigh(L, D, subset_by_index=[0, k-1])

        #Matrix U has as columns the eigenvectors
        U = eigenvectors

        #Applying k-means algorithm

        #Setting the random_state parameter to 1, to ensure reproducibility
across experiments.
        kmeans = KMeans(n_clusters=k, random_state=1)

        #U refers to the input data matrix
        kmeans.fit(U)

        #Obtain the cluster labels for each input data point
        cluster_idx = kmeans.labels_

        return cluster_idx
```

Η διαδικασία ξεκινά με τον υπολογισμό του Λαπλασιανού πίνακα L, όπως και στο spectral clustering. Ωστόσο, το πρόβλημα των ιδιοτιμών που επιλύεται εδώ είναι γενικευμένο, της μορφής:

$$\boldsymbol{Lx} = \lambda \boldsymbol{Dx}$$

Αυτό γίνεται με τη βοήθεια της συνάρτησης $eigh$, η οποία και πάλι επιστρέφει τις ιδιοτιμές/ιδιοδιανύσματα του συστήματος. Η παράμετρος $subset_by_index = [0, k-1]$ καθορίζει ότι μας ενδιαφέρουν τα k ιδιοδιανύσματα που αντιστοιχούν στις μικρότερες ιδιοτιμές.

Το αποτέλεσμα των ιδιοδιανυσμάτων αποθηκεύεται στον πίνακα U, και ακολουθεί εφαρμογή του $k - means$ πάνω στις γραμμές του πίνακα αυτού. Όπως και στις προηγούμενες συναρτήσεις, χρησιμοποιείται $random_state = 1$ για σταθερότητα στα αποτελέσματα.

Στο τέλος επιστρέφουμε έναν πίνακα με τις ετικέτες των pixels, που καθορίζουν σε ποιο cluster ανήκει το καθένα.

Παρακάτω παρατίθεται ο υλοποιημένος κώδικας της συνάρτησης $calculate_n_cut_value$:

```
def calculate_n_cut_value(
        affinity_mat: np.ndarray,
        cluster_idx: np.ndarray
) -> float:

    A = np.where(cluster_idx == 0)[0] #Nodes of the first cluster
    B = np.where(cluster_idx == 1)[0] #Nodes of the second cluster

    #assoc(A,V) is the sum of all the weights between the nodes of A (first
label) and the nodes of V (second label)

    assoc_A_A = np.sum(affinity_mat[np.ix_(A,A)])  #Sum of weights within cluster
A
    assoc_B_B = np.sum(affinity_mat[np.ix_(B,B)])  #Sum of weights within cluster
B

    assoc_A_V = np.sum(affinity_mat[A, :])
    assoc_B_V = np.sum(affinity_mat[B, :])

    N_assoc_A_B = (assoc_A_A/assoc_A_V) + (assoc_B_B/assoc_B_V)

    N_cut_A_B = 2 - N_assoc_A_B

    return N_cut_A_B
```

Συγκεκριμένα, εντοπίζονται οι δείκτες των κόμβων που ανήκουν στο cluster 0 και cluster 1 αντίστοιχα. Στη συνέχεια, υπολογίζονται:

- $assoc(A, A)$ και $assoc(B, B)$: τα αθροίσματα των βαρών εντός του κάθε cluster,
- $assoc(A, V)$ και $assoc(B, V)$: τα συνολικά βάρη μεταξύ των κόμβων κάθε cluster και όλων των κόμβων του γράφου.

Οι παραπάνω υπολογισμοί γίνανε με βάση τον τύπο:

$$assoc(A, V) = \sum_{u \in A, t \in V} W(u, t)$$

Με βάση αυτά, υπολογίζεται η ποσότητα:

$$N_{assoc}(A, B) = \frac{assoc(A, A)}{assoc(A, V)} + \frac{assoc(B.B)}{assoc(B, V)}$$

και στη συνέχεια η τελική τιμή του $Normalized\ Cut$, με βάση:

$$N_{cut}(A, B) = 2 - Nassoc(A, B)$$

Όσο μικρότερη είναι η τιμή της $N - cut$, τόσο «καλύτερη» θεωρείται η διαμέριση, καθώς υποδεικνύει υψηλή συνοχή εντός των clusters και μικρή συνδεσιμότητα μεταξύ τους.

Η συνάρτηση $n_cuts_recursive$ υλοποιεί την αναδρομική έκδοση της μεθόδου normalized cuts, με σκοπό τη σταδιακή διάσπαση του γράφου σε πολλαπλά υποσύνολα. Η βασική ιδέα είναι ότι ξεκινάμε με έναν αρχικό δυαδικό διαχωρισμό του γράφου και αποφασίζουμε αν θα συνεχιστεί η διαμέριση με βάση κριτήρια κατωφλίου.

```
def n_cuts_recursive(
        affinity_mat: np.ndarray,
        T1: int,
        T2: int,
) -> np.ndarray:

    cluster_idx = n_cuts(affinity_mat, 2)
    n_cut_value = calculate_n_cut_value(affinity_mat, cluster_idx)

    A = np.where(cluster_idx == 0)[0] #Nodes of the first cluster
    B = np.where(cluster_idx == 1)[0] #Nodes of the second cluster

    if(len(A) < T1 or len(B) < T1 or n_cut_value > T2):
        return cluster_idx

    #Split each cluster into two subclusters ans continue recursively
```

```
    cluster_idx_0 = n_cuts_recursive(affinity_mat[np.ix_(A, A)], T1, T2)
    cluster_idx_1 = n_cuts_recursive(affinity_mat[np.ix_(B, B)], T1, T2)

    cluster_idx_result = np.zeros_like(cluster_idx)

    cluster_idx_result[A] = cluster_idx_0
    cluster_idx_result[B] = cluster_idx_1 + np.max(cluster_idx_0) + 1

    return cluster_idx_result
```

Ο γράφος αρχικά διαχωρίζεται σε δύο clusters με χρήση της n_cuts.Υπολογίζεται η τιμή $Ncut$ για αυτόν τον διαχωρισμό μέσω της $calculate_n_cut_value$. Αν ένα από τα δύο clusters είναι μικρότερο από $T1$ κόμβους ή αν η τιμή του $Ncut$ υπερβαίνει το $T2$, τότε ο διαχωρισμός θεωρείται μη αποδοτικός και η αναδρομή σταματά.

Αν ικανοποιούνται οι συνθήκες, κάθε cluster χωρίζεται αναδρομικά με κλήση της ίδιας συνάρτησης στα αντίστοιχα υποπίνακα του $affinity\ matrix$.

Τέλος, οι ετικέτες συγχωνεύονται σωστά στον πίνακα $cluster_idx_result$, προσέχοντας να μη γίνει επικάλυψη των τιμών, αυξάνοντας κατάλληλα τις ετικέτες του δεύτερου υπο-διαχωρισμού.

Η συνάρτηση επιστρέφει έναν πίνακα ετικετών που περιέχει τον τελικό αναδρομικό διαχωρισμό του αρχικού γράφου.

4 Demos

4.1 demo1.py

Το πρώτο demo εξετάζει την λειτουργία της μεθόδου **spectral clustering** σε έναν έτοιμο affinity πίνακα (d1a) που παρέχεται στο αρχείο dip_hw_3.mat.

Συγκεκριμένα, το αρχείο .mat φορτώνεται με χρήση της $loadmat$. Η μεταβλητή $d1a$ περιέχει έναν συμμετρικό affinity πίνακα που αντιστοιχεί σε έναν πλήρως συνδεδεμένο γράφο. Η συνάρτηση $spectral_clustering$ εφαρμόζεται τρεις φορές, για $k = 2, k = 3\ και\ k = 4$, ώστε να παραχθούν διαφορετικά αποτελέσματα ομαδοποίησης. Η έξοδος του κάθε $spectral_clustering$ είναι ένας πίνακας ετικετών (labels), που δείχνει σε ποιο cluster ανήκει κάθε κόμβος του γράφου.

Έπειτα από την εφαρμογή του κώδικα, τα αποτελέσματα είναι τα παρακάτω:

```
Labels for k=2: [1 1 1 1 0 0 0 0 0 0 0 0]
Labels for k=3: [2 2 2 2 0 0 0 0 1 1 1 1]
Labels for k=4: [2 2 2 2 0 0 3 3 1 1 1 1]
```

- $k = 2$

Οι πρώτοι τέσσερις κόμβοι κατατάσσονται στο cluster 1 και οι υπόλοιποι οκτώ στο cluster 0. Αυτό υποδηλώνει έναν καθαρό διαχωρισμό του γράφου σε δύο περιοχές με διαφορετική εσωτερική συνδεσιμότητα.

- $k = 3$

Εμφανίζονται τρεις ομάδες: οι πρώτοι τέσσερις κόμβοι στο cluster 2, οι επόμενοι τέσσερις στο cluster 0 και οι τελευταίοι τέσσερις στο cluster 1. Εδώ παρατηρείται μια πιο λεπτομερής κατάτμηση.

- $k = 4$

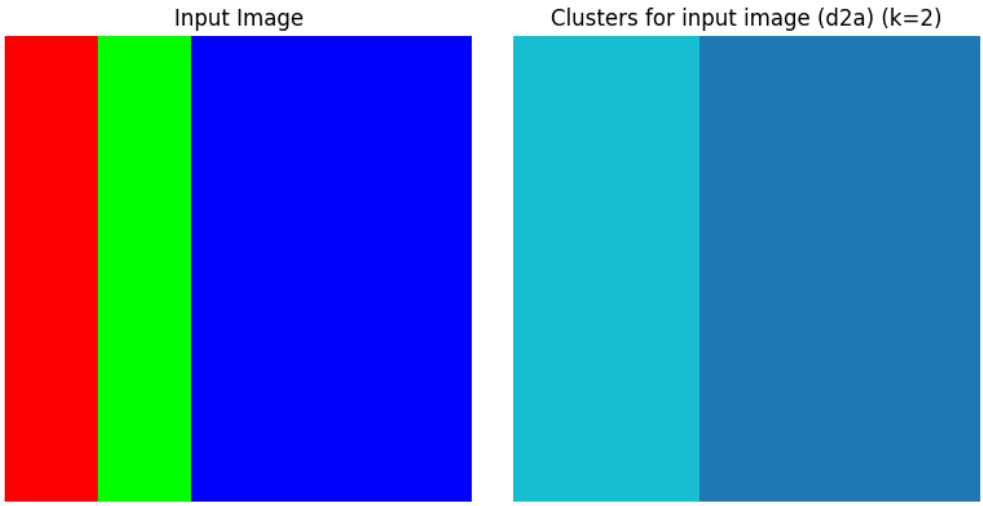
Εδώ, η ταξινόμηση γίνεται ακόμη πιο σύνθετη, καθώς τα μεσαία pixels χωρίζονται περαιτέρω.

4.2 demo2.py

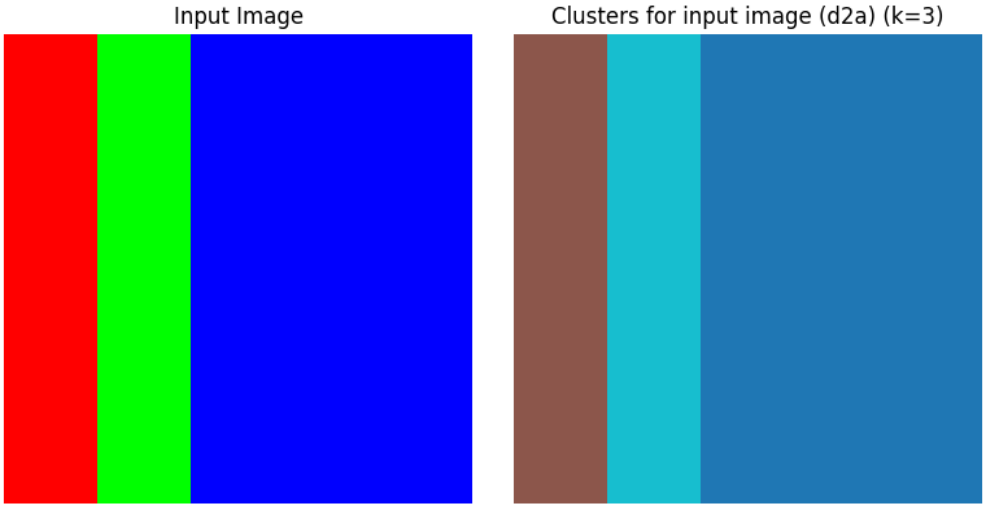
Στο demo αυτό εφαρμόζεται spectral clustering σε δύο έγχρωμες εικόνες (d2a και d2b) με στόχο την οπτική κατάτμησή τους σε 2, 3 και 4 περιοχές. Πριν την εφαρμογή της μεθόδου, κάθε εικόνα μετατρέπεται πρώτα σε γράφο με τη χρήση της image_to_graph.

Αποτελέσματα για την εικόνα εισόδου d2a

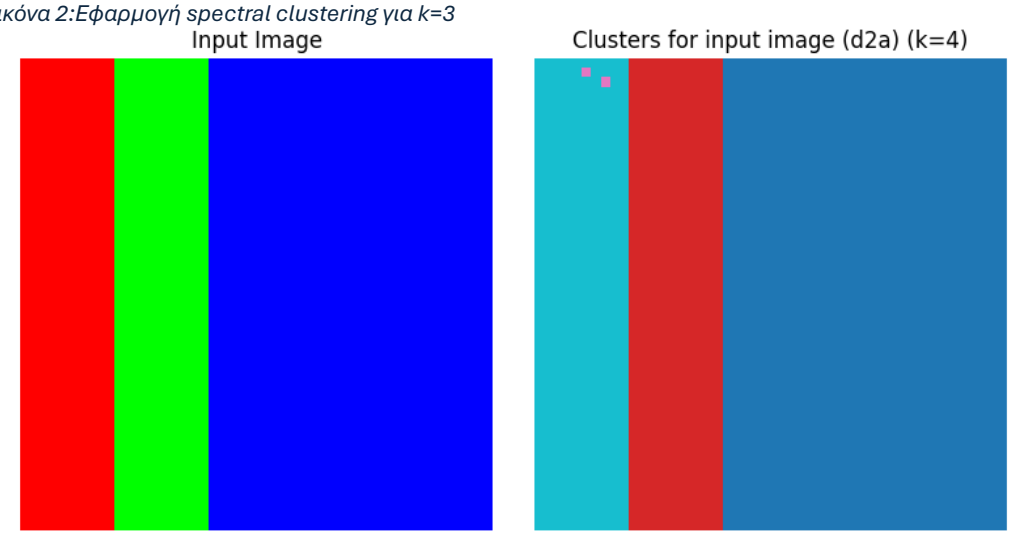
Αρχικά, η εικόνα d2a αποτελείται από τρεις καθαρές χρωματικές ζώνες: κόκκινο, πράσινο και μπλε, κατανεμημένες κάθετα. Ο στόχος εδώ είναι να δούμε αν το $spectral\ clustering$ μπορεί να εντοπίσει αυτές τις διακριτές περιοχές. Παρακάτω βλέπουμε τα αποτελέσματα για εφαρμογή σε k περιοχές και όπως είπαμε και παραπάνω για $k = 2,3,4$.



Εικόνα 1:Εφαρμογή spectral clustering για k=2



Εικόνα 2:Εφαρμογή spectral clustering για k=3



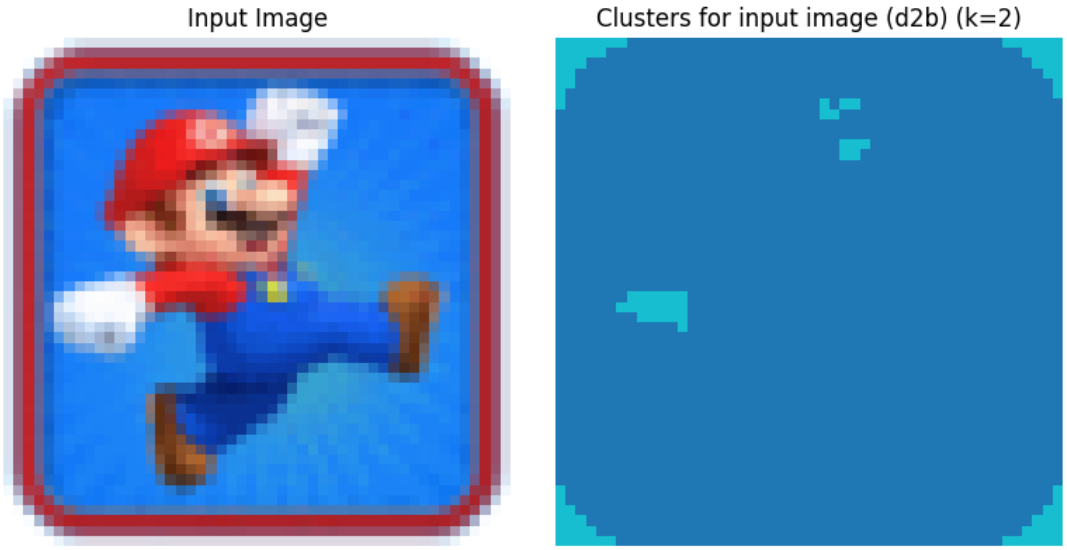
Εικόνα 3:Εφαρμογή spectral clustering για k=4

$\boldsymbol{k = 2}$.Η μέθοδος διαχωρίζει την εικόνα σε δύο ομάδες. Η κόκκινη και η πράσινη περιοχή ομαδοποιούνται μαζί, ενώ η μπλε διαχωρίζεται ως ξεχωριστό cluster.

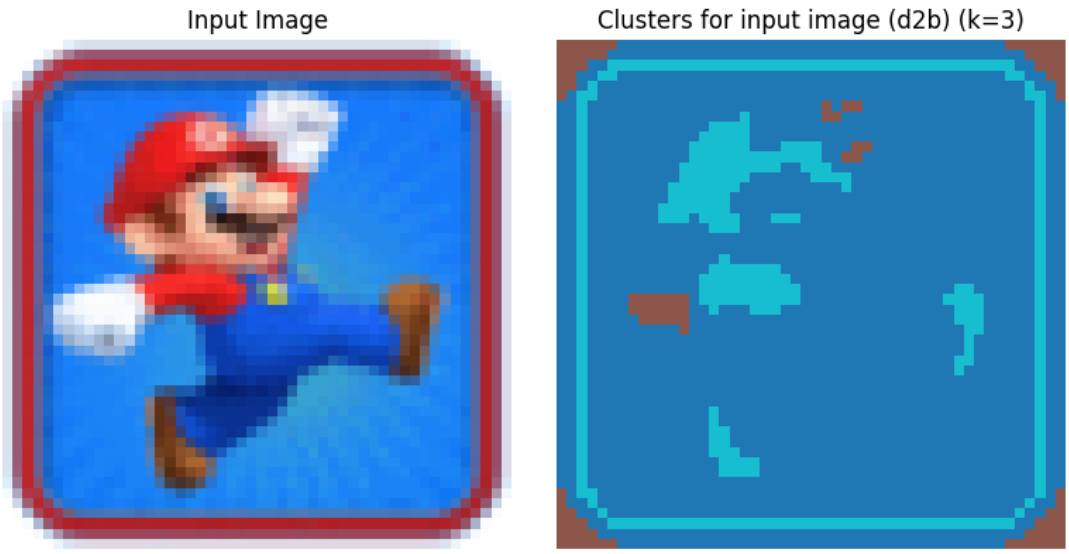
$\boldsymbol{k = 3}$.Παρατηρείται πλήρης αντιστοίχιση των clusters με τις τρεις χρωματικές ζώνες της εικόνας. Η μέθοδος εντοπίζει με ακρίβεια την κόκκινη, πράσινη και μπλε περιοχή ως ξεχωριστές ομάδες. Το αποτέλεσμα αυτό είναι το επιθυμητό και αποδεικνύει την αποτελεσματικότητα της μεθόδου σε καλά διαχωρισμένες, ομοιογενείς περιοχές.

$\boldsymbol{k = 4}$.Εμφανίζεται μία επιπλέον ομάδα που εντοπίζεται κυρίως εντός της κόκκινης (αρχικής) περιοχής, πιθανώς λόγω μικρών διαφορών φωτεινότητας ή αριθμητικών σφαλμάτων. Ο διαχωρισμός αυτός δεν προσθέτει χρήσιμη πληροφορία, καθώς οδηγεί σε υπερβολική κατάτμηση.

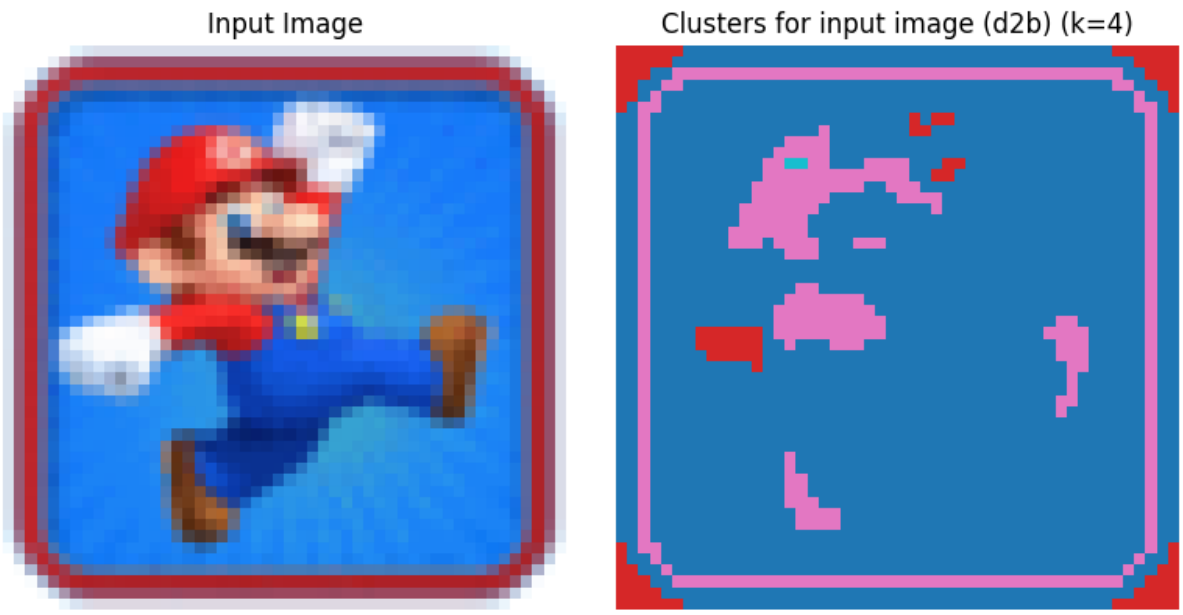
Αποτελέσματα για την εικόνα εισόδου d2b



Εικόνα 4:Εφαρμογή spectral clustering για k=2



Εικόνα 5:Εφαρμογή spectral clustering για k=3



Εικόνα 6:Εφαρμογή spectral clustering για k=4

$\boldsymbol{k = 2}$.Ο αλγόριθμος έχει καταφέρει να ξεχωρίσει το φόντο και κάποια ελάχιστα σημεία από τα γάντια του χαρακτήρα. Το segmentation είναι γενικό, το οποίο είναι αναμενόμενο λόγω της αρκετής πληροφορίας της αρχικής εικόνας και της επιλογής $k = 2$.

$\boldsymbol{k = 3}$.Τα αποτελέσματα εμφανίζουν περισσότερη λεπτομέρεια, με το εσωτερικό του χαρακτήρα να διαχωρίζεται περαιτέρω (π.χ. καπέλο, πόδια, χέρια). Η μέθοδος αρχίζει να εντοπίζει υποπεριοχές με παρόμοια χρωματικά χαρακτηριστικά.

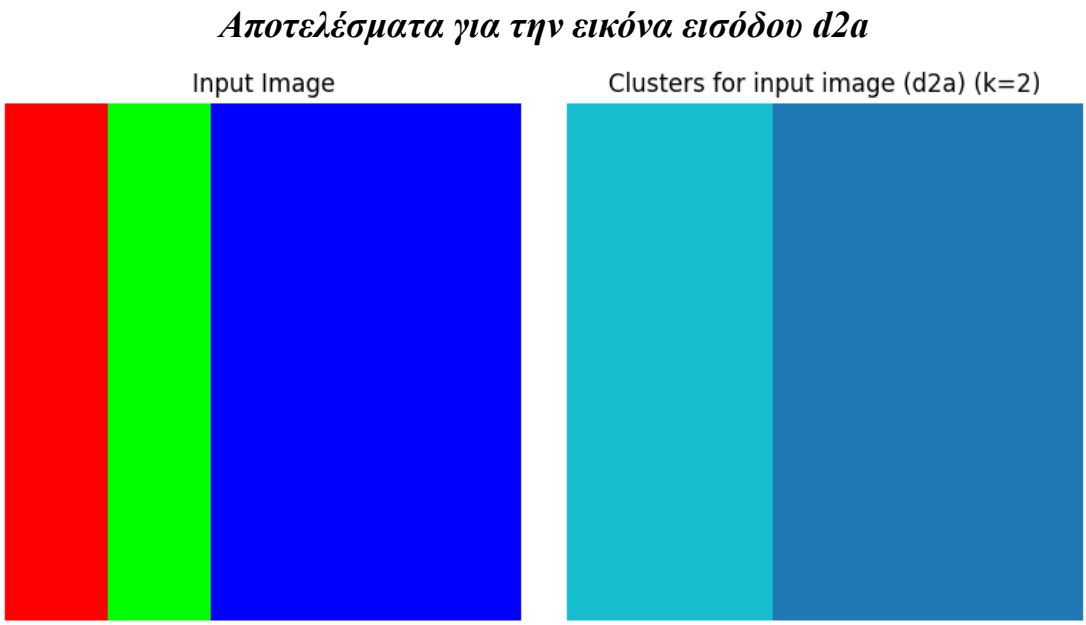
$\boldsymbol{k = 4}$.Ο διαχωρισμός παραμένει παρόμοιος με την περίπτωση του $k = 3$. Αυτό είναι πολύ πιθανό να οφείλεται στη παρουσία θολώματος της εικόνας, καθώς η μετάβαση των χρωματικών τιμών μεταξύ περιοχών είναι σταδιακή. Αντί για απότομα όρια μεταξύ αντικειμένων, το blur δημιουργεί ομαλά και ασαφή όρια.

Παρακάτω βλέπουμε και τις εκτυπώσεις του κώδικα όσο αφορά τις ετικέτες:

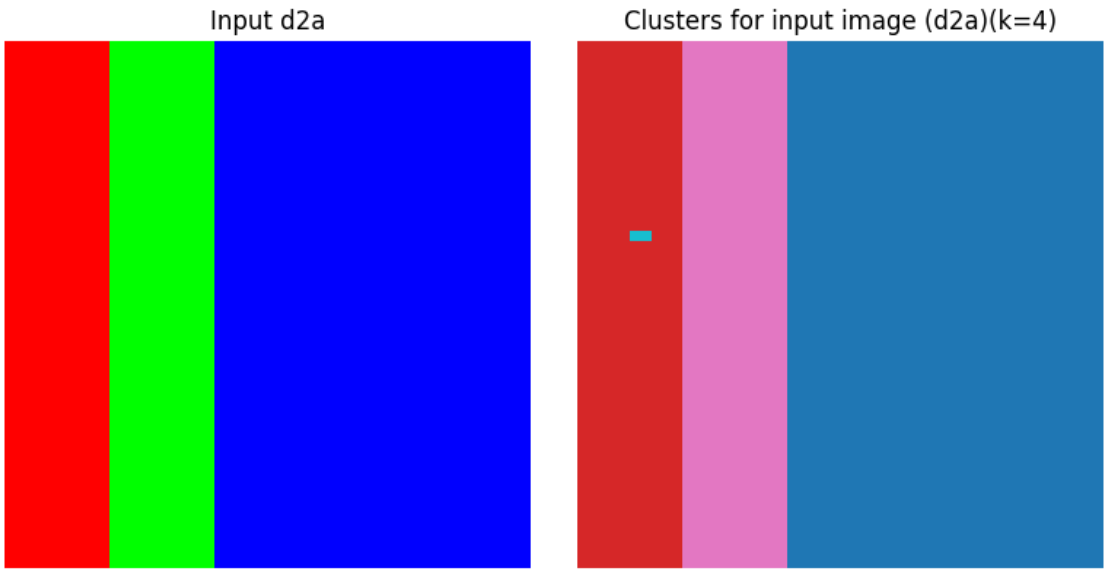
```
Labels for d2a with k=2: [1 1 1 ... 0 0 0]
Labels for d2a with k=3: [1 1 1 ... 0 0 0]
Labels for d2a with k=4: [3 3 3 ... 0 0 0]
Labels for d2b with k=2: [1 1 1 ... 1 1 1]
Labels for d2b with k=3: [1 1 1 ... 1 1 1]
Labels for d2b with k=4: [1 1 1 ... 1 1 1]
```

4.3 demo3a.py

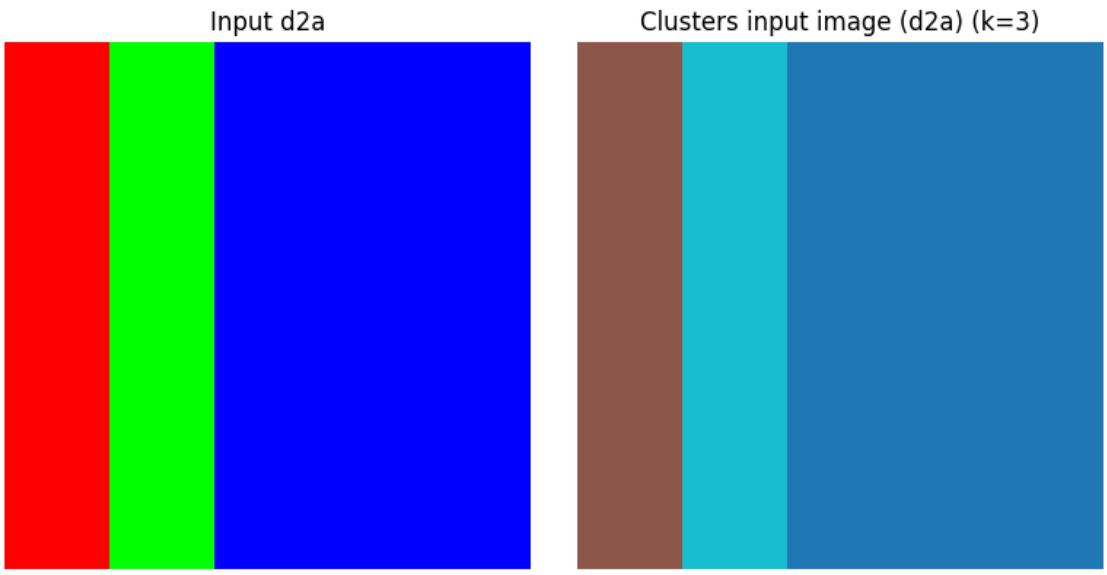
Στο demo αυτό ομαδοποιούμε τα nodes των εικόνων εισόδου με την μη-αναδρομική εκδοχή της μεθόδου *n*cuts, με στόχο και πάλι την οπτική κατάτμησή τους σε 2, 3 και 4 περιοχές



Εικόνα 7:Εφαρμογή N-cuts για k = 2



Εικόνα 9:Εφαρμογή N-cuts για k = 4

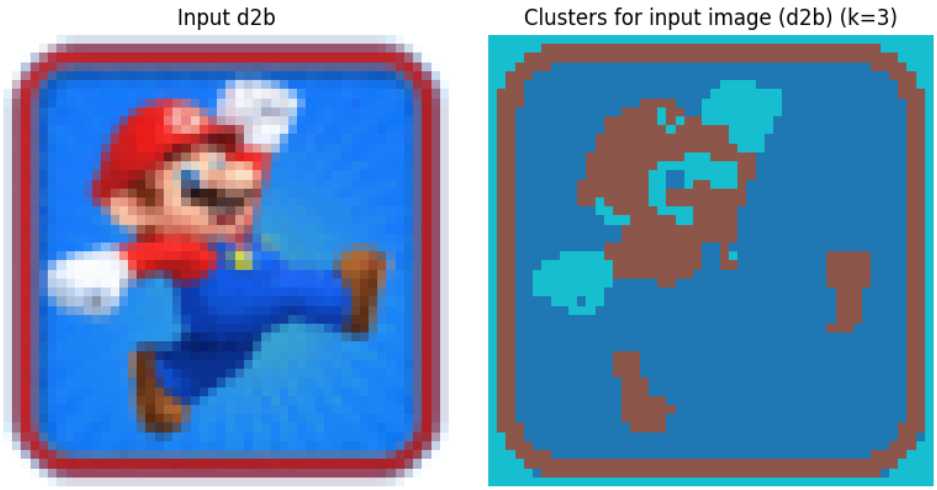


Εικόνα 8:Εφαρμογή N-cuts για k = 3

- $\boldsymbol{k = 2}$: Η μέθοδος Normalized Cuts χωρίζει την εικόνα σε δύο clusters, διαχωρίζοντας το μπλε από τα υπόλοιπα. Τα αποτελέσματα, συμβαδίζουν με αυτά του spectral clustering.
- $\boldsymbol{k = 3}$: Και εδώ, η κατάτμηση είναι ιδανική. Οι τρεις λωρίδες εντοπίζονται ξεκάθαρα, αποδεικνύοντας ότι η μέθοδος N-Cuts μπορεί να αποδώσει καλά όταν τα clusters είναι συμπαγή και καλά διαχωρισμένα.
- $\boldsymbol{k = 4}$: Σε αυτή την περίπτωση, η μέθοδος προσπαθεί να διαχωρίσει περαιτέρω τις ήδη καλά ορισμένες περιοχές, οδηγώντας σε υπερκατάτμηση. Το τέταρτο cluster εμφανίζεται σε περιορισμένες τοπικές περιοχές, χωρίς να προσθέτει ουσιαστική πληροφορία. Η συμπεριφορά είναι αντίστοιχη με την παρατηρούμενη και στο spectral clustering, όπου και εκεί το τέταρτο cluster ήταν περιττό και δεν είχε φυσικό νόημα.

Αποτελέσματα για την εικόνα εισόδου d2b

Εικόνα 10:Εφαρμογή N-cuts για k = 2



Εικόνα 11:Εφαρμογή N-cuts για k = 3

Εικόνα 12:Εφαρμογή N-cuts για k = 4

$\boldsymbol{k = 2}$: Η μέθοδος καταφέρνει να διαχωρίσει σε γενικές γραμμές τον χαρακτήρα από το φόντο, Σε σχέση με το spectral, παρατηρείται πιο έντονος διαχωρισμός σε συγκεκριμένα σημεία.

$\boldsymbol{k = 3}$ Εντοπίζονται περισσότερες εσωτερικές δομές εντός του χαρακτήρα, όπως τμήματα του προσώπου ή της στολής. Όπως και πριν, σε σχέση με το spectral, παρατηρείται αρκετά πιο έντονος διαχωρισμός.

$\boldsymbol{k = 4}$: Παρατηρούμε αρκετά καλύτερα αποτελέσματα από αυτά του spectral clustering. Σε σχέση και με το $k = 3$, με αυτή τη μέθοδο έχουμε διαφορά στις δυο κατατμήσεις.

Παρακάτω βλέπουμε και τις εκτυπώσεις του κώδικα όσο αφορά τις ετικέτες:

```
Labels for d2a with k=2: [1 1 1 ... 0 0 0]
Labels for d2b with k=2: [1 1 1 ... 1 1 1]
Labels for d2a with k=3: [1 1 1 ... 0 0 0]
Labels for d2b with k=3: [2 2 2 ... 2 2 2]
Labels for d2a with k=4: [1 1 1 ... 0 0 0]
Labels for d2b with k=4: [1 1 1 ... 1 1 1]
```

4.4 demo3b.py

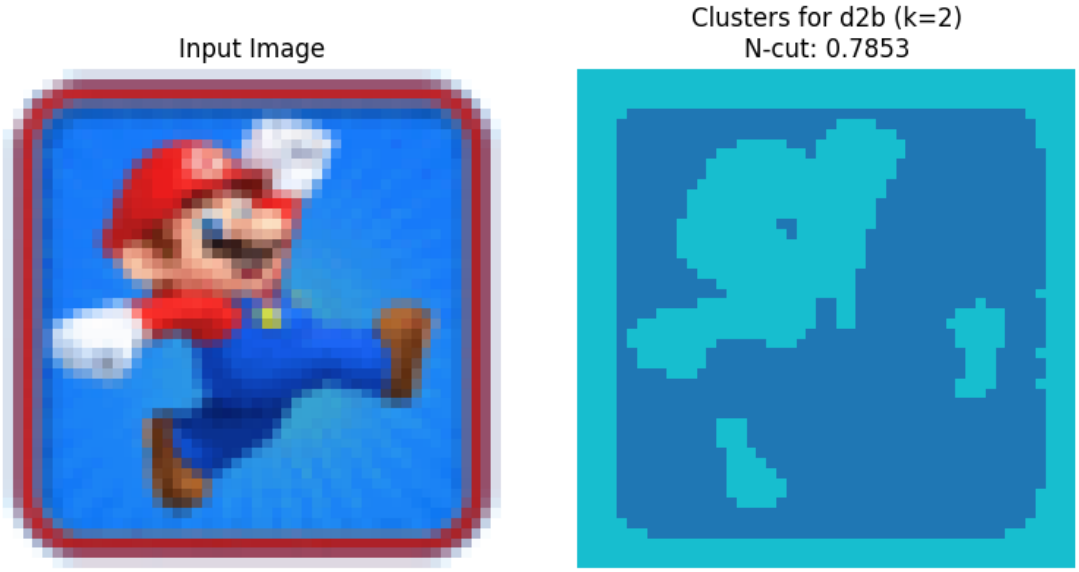
Σε αυτή τη φάση, εφαρμόσαμε την αναδρομική συνάρτηση $n_cuts_recursive$ μόνο για ένα βήμα, δηλαδή διαχωρισμό του γράφου σε δύο μέρη, τόσο για την εικόνα $d2a$ όσο και για την $d2b$. Στη συνέχεια, υπολογίσαμε και παρουσιάσαμε την τιμή της μετρικής $N - cut$ για κάθε περίπτωση.

Αποτελέσματα για την εικόνα εισόδου d2a

Εικόνα 13:Εφαρμογή N-cuts Recursive για k = 2

Η μέθοδος διαχώρισε με σαφήνεια την εικόνα σε δύο ομάδες, διαχωρίζοντας την κόκκινη και πράσινη στήλη από το μπλε φόντο. Επίσης, η τιμή της μετρικής N-cut είναι 0.5092, η οποία είναι χαμηλή, δείχνοντας ότι η τομή έγινε σε κατάλληλο σημείο με βάση τη δομή του γράφου. Συγκριτικά με τη spectral clustering, οι δύο μέθοδοι είχαν ίδια απόδοση ως προς τον τελικό διαχωρισμό.

Αποτελέσματα για την εικόνα εισόδου d2b



Εικόνα 14:Εφαρμογή N-cuts Recursive για k = 2

Ο διαχωρισμός εντοπίζει κυρίως το σώμα του χαρακτήρα από το μπλε φόντο. Η τιμή της μετρικής N-cut είναι 0.7853, σημαντικά υψηλότερη, γεγονός που υποδεικνύει λιγότερο «φυσικό» διαχωρισμό, πιθανόν λόγω της πολυπλοκότητας και θόλωσης της εικόνας. Συγκριτικά τώρα, το spectral clustering δεν είχε τόσο καλά αποτελέσματα, η N-cuts κατέγραψε πιο εντοπισμένες περιοχές του αντικειμένου, αποτυπώνοντας καλύτερα το σχήμα του χαρακτήρα.

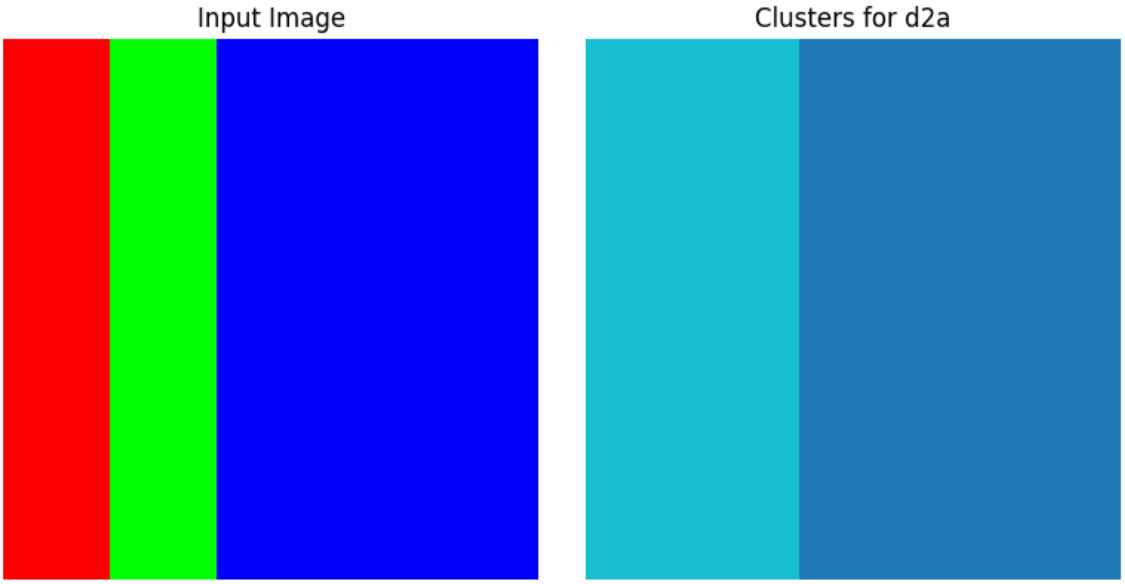
Παρακάτω βλέπουμε και τις εκτυπώσεις του κώδικα όσο αφορά τις ετικέτες:

```
Labels for d2a with k=2: [1 1 1 ... 0 0 0]
N-cut metric for d2a with k=2: 0.509237924200272
Labels for d2b with k=2: [1 1 1 ... 1 1 1]
N-cut metric for d2a with k=2: 0.7852852891229902
```

4.5 demo3c.py

Για το τελευταίο demo, εφαρμόσαμε τη συνολική, αναδρομική εκτέλεση της μεθόδου Normalized Cuts (N-cuts) στις εικόνες d2a και d2b, χρησιμοποιώντας κατώφλια T1 = 5 και T2 = 0.2.

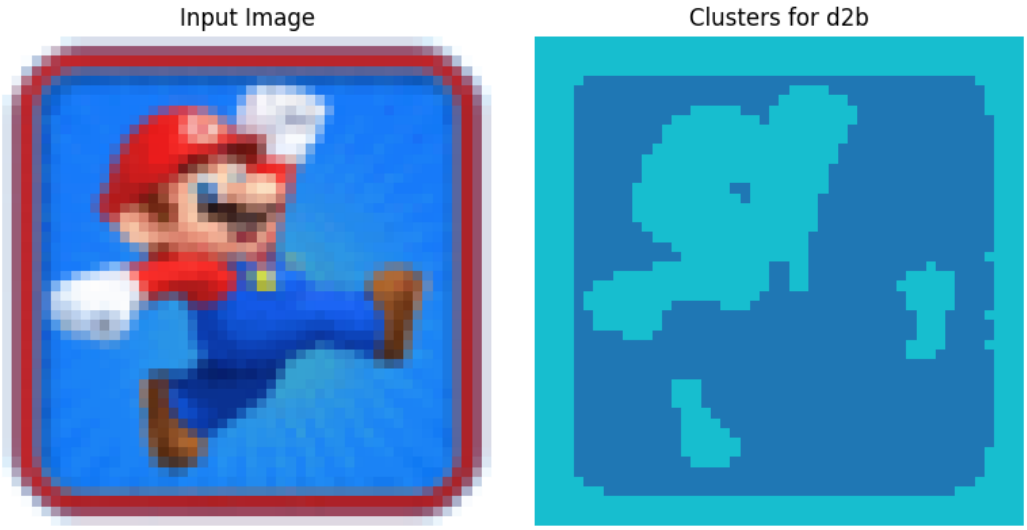
Αποτελέσματα για την εικόνα εισόδου d2a



Εικόνα 15: Εφαρμογή Completed N-cuts Recursive για k = 2

Τα αποτελέσματα και αυτής της μεθόδου είναι ίδια με αυτά της $spectral\ clustering$ και $non-recursive\ N-cuts$.

Αποτελέσματα για την εικόνα εισόδου d2b



Τα αποτελέσματα της μεθόδου είναι ίδια με της μη αναδρομικής $N-cuts$ για $k\ =\ 2$ και οι διαφορές με τη μέθοδο $spectral\ clustering$ είναι οι ίδιες όπως με την $N-cuts$ για $k\ =\ 2$.